慢在哪，以及先动哪

6,529 条记录、1,528 次调用重算 · 2026-11-01 · 芝士

一、先更正一个数

上一版写「每次调用约 2.7 秒固定开销」。**这个数是拟合出来的，不是实测值**。我用同样的方法精确复现出 2.71 秒，但它被两件事压低了：终点取的是第一个块（把这块自己的生成时间排除在外），且前 721 次有四成落在 10-23 那天——那天整体只要 0.9 秒。

二、实测是什么样

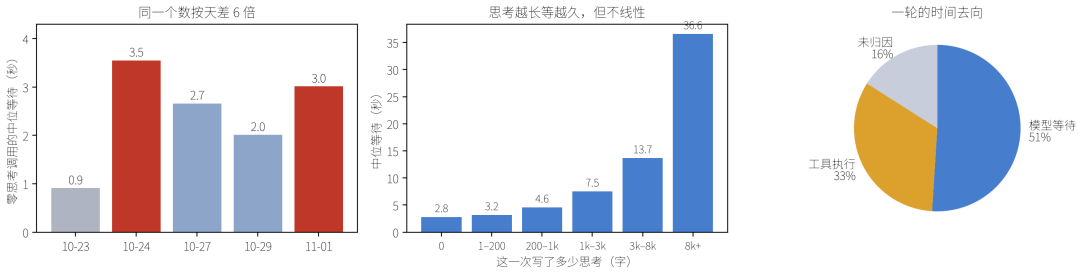


图 1　左：零思考调用的中位等待，按天拆开差 6 倍。中：写多少思考等多久。右：一轮的时间去向。

零思考的调用（几乎不生成内容）中位 **2.80 秒**，最近两天 **3.01 秒**。扣掉首块自己吐的字（约 0.5 秒），真正「不算内容」的部分是 **1.5–2.2 秒**。

它不固定：同样的工具、同样的参数规模，10-23 是 0.47–0.60 秒，10-24 是 3.17 秒，**差 6 倍**。

思考越长等越久，但不线性：0 字 2.8 秒、1–3 千字 7.5 秒、8 千字以上 36.6 秒。边际成本在 1000 字以上才明显起来。

三、三个之前不知道的

长尾吃掉四分之一：超过 30 秒的调用只占 **3.3%**，但吃掉全部模型等待的 **25%**。其中 7 次是零思考、参数几百字，却等了 66–105 秒——最长那次参数只有 338 字，是一个发消息的调用。

工具执行占三分之一：中位 0.48 秒，但占一轮墙钟的 **33%**。最慢的 1% 吃掉其中 37%；WebFetch 中位 12 秒，6 次命令行卡在 120 秒超时。

跟上下文长度没关系：用 turn 内累计字符做代理，零思考调用的等待从 0.91 到 0.99 秒几乎不变，全量回归 r^2=0.01。

所以「预填充越来越长的上下文导致慢」在我们数据里没有支持。

四、要做的事，按顺序

1. 先加首字埋点（已做）

网关那套日志本来记「请求发出」「响应头」「流结束」三个点，缺中间那个。已补上「上游第一个字节」。之后那 1.5–2.2 秒能拆成两段：连接与排队、等待首字。

还测不到的：预填充与排队的边界（要上游配合）、token 数与缓存命中（要另一个埋点）。

2. 沙箱镜像补工具链

`backend/sandbox/Dockerfile.private` 里只装了 **python3、bash、ca-certificates、ripgrep**，没有编译器、没有 poppler。后果实测过：一轮 21 次命令行调用里 10 次在补依赖，最后卡在共享库加载失败，**整轮零产出**。

这一条不涉及取舍，装上就行。

3. 读文件去重

同一份 `.typ` 在 4 轮里被读了 **12 次**、同一张 PNG 被读 6 次，每次都全文重进上下文。仓库里查不到「本会话读过、没变就跳过」的机制。

4. 平台提醒不占轮次

「状态 10 分钟没动过」这类提醒，阈值 600 秒，每句都作为一条消息开一整轮。本会话出现十几次。

五、别人怎么做的（有实测的）

并行工具调用有硬证据。LLMCompiler（ICML 2024）Table 1，同一实验、同一模型，OpenAI 并行函数调用 vs 串行 ReAct：**20.47 秒 → 7.42 秒**（2.76 倍）、7.12 → 4.42（1.61 倍）、35.90 → 19.29（1.86 倍）。

但别当万能药：有生产实测说，盲目开并行 p50 只降 **18%**，浪费调用反而从 0.3% 涨到 7.1%。

六、所以先做什么

先让埋点跑一周。现在能确定的只有三件事：它不固定、它有个吃钱的长尾、它跟上下文长度无关。在这三条之上谈优化都是猜——上面第 2 到 4 条是查实了缺的，可以并行做，但它们都碰不到那 25% 的长尾和 33% 的工具时间。

口径：本任务 2026-10-23 至 11-01 的 6,529 条记录、1,528 次调用。时间戳是块写完的时刻，相邻两块的时间差只和后一块大小相关（r^2=0.80–0.97）。LLMCompiler 见 arXiv:2312.04511 Table 1；生产反例见 callsphere.ai。第 2–4 条的代码依据：`Dockerfile.private`、`prompt.py`、`core/config.py:908`。